Practice Manual 2

Build a Database Website with Django

Introduction to Python | Project: My Book Library

You will build	A book library website that stores books in a database, shows them in a table, and lets you add new books using a form
You need	A laptop with Python 3.12 or newer, VS Code, and a web browser
Time	About 2 hours
Level	Beginner. Completing Practice Manual 1 first is strongly recommended
Names used	Folder: `practice2_library` Project: `config` App: `books`

What Is New in This Manual?

In Practice 1, the content of every page was typed into HTML files. Real websites do not work like that. Their information (products, students, marks, books) is kept in a **database**, and the website reads it and shows it on the page. When new data is added, the website updates itself without anyone editing HTML.

A database stores information in an organised way. Picture an Excel sheet: the columns have headings (Title, Author, Year) and every book is one row. Django lets you describe that sheet using a Python class called a **model**. You never write database commands yourself, because Django does that work for you.

Spreadsheet idea	Django word	In this project
A sheet	**Model** (a Python class)	`Book`
A column heading	**Field**	`title`, `author`, `year_published`
One row	**Record** (an object)	One single book
The Excel file	**Database**	The file `db.sqlite3`

You will use five building blocks. The first three are new or extended from Practice 1:

Building block	Its job	Where it lives
Model	Describes the data to be stored	`models.py`
URL	Connects a web address to a view	`urls.py`
View	Gets data from the database and hands it to a template	`views.py`
Template	Shows the data as an HTML page	`templates` folder
Form	Collects input from the visitor and saves it	`forms.py`

How to Use This Manual

Follow the steps in order and do not skip any. Each step tells you **what to do** and, in the blue box, **why you are doing it**. The coloured boxes mean:

> **WHY WE DO THIS** Explains the reason behind a step. Read it: understanding the reason is how you learn.

> **CHECKPOINT** Tells you what you should see before moving to the next step. If you do not see it, stop and fix it first.

> **TIP** A helpful shortcut or extra information.

> **WATCH OUT** A common mistake. Read this before you continue.

Three rules that prevent most problems:

1. **Type carefully.** Python is sensitive to spaces at the start of a line (indentation), to quotes, and to commas.
2. **Save every file.** Turn on Auto Save in Step 1. Django can only use code that has been saved.
3. **Read the last line of any error message first.** It usually names the problem. Common errors are listed at the end of this manual.

Part A: Setting Up the Project

If you completed Practice 1, this part will feel familiar. Use a **new folder** for this project. Do not reuse the folder from Practice 1.

Step 1: Create a Folder and Open It in VS Code

1. On your Desktop, create a new folder named `practice2_library`.
2. Open VS Code. Click **File** → **Open Folder**, select `practice2_library`, and click **Open**.
3. Open the terminal: click **Terminal** → **New Terminal**.
4. Turn on Auto Save: click **File** → **Auto Save** and make sure a tick appears beside it.

> **WHY WE DO THIS** Django only sees files that have been saved. Auto Save removes the most common beginner problem: code that looks right on screen but was never saved.

> **TIP** A small white dot on a file's tab means the file has unsaved changes. Press Cmd + S (Mac) or Ctrl + S (Windows) to save.

Step 2: Create the Virtual Environment and Install Django

Type these commands in the terminal, one at a time. Use the column that matches your computer.

Task	Mac	Windows
Create the environment	`python3 -m venv venv`	`python -m venv venv`
Activate it	`source venv/bin/activate`	`venv\Scripts\activate`

When `(venv)` appears at the start of the terminal line, install Django. From now on, use `python` on both Mac and Windows:

TERMINAL **Type in the terminal**

```
pip install django
python -m django --version
```

> **WHY WE DO THIS** The virtual environment is a private toolbox for this project. Django is installed inside it so it does not affect other projects on your laptop.

> **CHECKPOINT** `(venv)` shows at the start of the terminal line, and a version number is printed by the second command.

> **WATCH OUT** Every time you reopen VS Code, activate the environment again. If `(venv)` is missing, run the Activate command. On Windows PowerShell, if activation is blocked, run `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass` and then activate again.

Step 3: Tell VS Code to Use the Virtual Environment

1. Press **Cmd + Shift + P** (Mac) or **Ctrl + Shift + P** (Windows).
2. Type `Python: Select Interpreter` and press Enter.
3. Choose the option that mentions `venv`.

WHY WE DO THIS This lets VS Code find Django, so it does not underline correct code in red.

Step 4: Create the Project and the App

TERMINAL **Type in the terminal**

```
django-admin startproject config .
python manage.py startapp books
```

WHY WE DO THIS The first command creates the project (the whole website). The dot at the end means “right here in this folder”. The second command creates an app named `books`, which will handle everything about books. Think of the project as a college building and the app as one department in it.

CHECKPOINT The Explorer panel shows the folders `config` and `books`, and the file `manage.py`.

Step 5: Register the App

Open `config/settings.py`, find `INSTALLED_APPS`, and add `'books',` at the end of the list:

FILE **config/settings.py**

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'books',
]
```

WHY WE DO THIS Django only uses the apps listed in `INSTALLED_APPS`. This line introduces our `books` app to the project.

WATCH OUT Keep the quotes and the comma after `'books'`.

Part B: Storing Data with a Model

This is the new and most important part. We describe what a book looks like, let Django build the database table, and add some books.

Step 6: Describe the Book Table

Open `books/models.py`, delete everything, and type:

FILE **books/models.py**

```
from django.db import models

class Book(models.Model):
    title = models.CharField(max_length=200)
    author = models.CharField(max_length=100)
    year_published = models.IntegerField()
    is_available = models.BooleanField(default=True)

    def __str__(self):
        return self.title
```

Line	What it means
`class Book(models.Model):`	Creates a model named Book. Django will build a table for it.
`title = models.CharField(max_length=200)`	A column for short text, up to 200 characters.
`author = models.CharField(max_length=100)`	A column for short text, up to 100 characters.
`year_published = models.IntegerField()`	A column for whole numbers, such as 1999.
`is_available = models.BooleanField(default=True)`	A yes/no column. A new book starts as available.
`def __str__(self)`	Tells Django to show a book's title whenever it needs a name for it.

WHY WE DO THIS A **model** is a Python class that describes one kind of thing you want to store. Each field becomes a column in the database table. Django also adds an `id` column automatically, so every book gets its own number.

Step 7: Create the Database Table

TERMINAL **Type in the terminal**

```
python manage.py makemigrations
python manage.py migrate
```

The first command should print something like this:

```
Migrations for 'books':
  books/migrations/0001_initial.py
    + Create model Book
```

WHY WE DO THIS Writing the model is like drawing a plan. The table does not exist yet. `makemigrations` prepares the instructions, and `migrate` carries them out and builds the table inside the file `db.sqlite3`. Whenever you change `models.py` later, run both commands again.

CHECKPOINT The `migrate` command prints a list of lines ending with `OK`, and a file named `db.sqlite3` appears in the Explorer panel.

Step 8: Create an Admin Account

TERMINAL **Type in the terminal**

```
python manage.py createsuperuser
```

Django asks for a username, an email address (you may leave it blank by pressing Enter), and a password twice.

> **WHY WE DO THIS** Django comes with a ready-made admin website where you can add and edit data. You need an admin account to log in to it.

> **WATCH OUT** The password is invisible while you type. This is normal. If Django says the password is too simple, it asks "Bypass password validation?". For this practice you may type `y` and press Enter. Remember your username and password.

Step 9: Show the Book Model in the Admin Site

Open `books/admin.py`, delete everything, and type:

FILE **books/admin.py**

```
from django.contrib import admin
from .models import Book

class BookAdmin(admin.ModelAdmin):
    list_display = ('title', 'author', 'year_published', 'is_available')

admin.site.register(Book, BookAdmin)
```

> **WHY WE DO THIS** The admin site does not show your models automatically. `register` tells Django: "let me manage Book records here". `list_display` chooses which columns appear in the admin list, so you see full details instead of only titles.

Step 10: Run the Server and Add Books

TERMINAL **Type in the terminal**

```
python manage.py runserver
```

BROWSER **Type in the address bar**

```
http://127.0.0.1:8000/admin/
```

1. Log in with the username and password from Step 8.
2. Click **Books**, then **Add Book** (top right).
3. Fill in the fields and click **Save**. Repeat for the books in the table below (or use your own favourites).
4. For one book, untick **Is available** before saving.

Title	Author	Year
Wings of Fire	A. P. J. Abdul Kalam	1999
The Alchemist	Paulo Coelho	1988
Python Crash Course	Eric Matthes	2015
Malgudi Days	R. K. Narayan	1943
Ikigai	Hector Garcia and Francesc Miralles	2016

CHECKPOINT The admin list shows your books in columns: title, author, year, and availability.

TIP Leave the server running. It restarts automatically whenever you save a file. To stop it, click in the terminal and press **Ctrl + C**.

Part C: Showing Data on a Web Page

The books are in the database. Now we build a page for visitors that reads them and displays them in a table.

Step 11: Write the View

Open `books/views.py`, delete everything, and type:

FILE `books/views.py`

```
from django.shortcuts import render
from .models import Book

def book_list(request):
    books = Book.objects.all()
    return render(request, 'books/book_list.html', {'books': books})
```

- `from .models import Book` brings in the model we created in Step 6.
- `Book.objects.all()` asks the database for every book. The result is stored in `books`.
- `{'books': books}` is a bag of data handed to the template. The word `'books'` is the name the template will use.

WHY WE DO THIS The view is the middleman. It fetches data from the database and gives it to the template, which decides how to display it.

Step 12: Create the App's URL File

Right-click the `books` folder, choose **New File**, name it `urls.py`, and type:

FILE `books/urls.py`

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.book_list, name='book_list'),
]
```

WHY WE DO THIS This connects the main address of the site (the empty quotes) to the `book_list` view. The `name` is a nickname we will use for links.

WATCH OUT The list must be named exactly `urlpatterns` (all lowercase, ending with s).

Step 13: Connect It to the Main URL File

Open `config/urls.py`, delete everything, and type:

FILE **config/urls.py**

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('books.urls')),
]
```

WHY WE DO THIS Django checks `config/urls.py` first. The `include('books.urls')` line says: “for everything else, look in the books app’s own URL file”. The admin address still works because it is listed first.

Step 14: Create the Template

1. Right-click the `books` folder, choose **New Folder**, and name it `templates`.
2. Inside `templates`, create a folder named `books`.
3. Inside that inner `books` folder, create a new file named `book_list.html`.

The final path must be `books/templates/books/book_list.html`. Type:

FILE **books/templates/books/book_list.html**

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>My Book Library</title>
</head>
<body>
    <h1>My Book Library</h1>
    <p>Total books: {{ books|length }}</p>

    <table border="1" cellpadding="8">
        <tr>
            <th>No.</th>
            <th>Title</th>
            <th>Author</th>
            <th>Year</th>
            <th>Status</th>
        </tr>
        {% for book in books %}
        <tr>
            <td>{{ forloop.counter }}</td>
            <td>{{ book.title }}</td>
            <td>{{ book.author }}</td>
            <td>{{ book.year_published }}</td>
            <td>{{ book.is_available|yesno:"Available,Issued" }}</td>
        </tr>
        {% empty %}
        <tr>
            <td colspan="5">No books yet. Add some in the admin page.</td>
        </tr>
        {% endfor %}
    </table>
</body>
</html>
```

Template code	What it does
`{{ books|length }}`	Displays how many books there are.
`{% for book in books %}` ... `{% endfor %}`	Repeats the row once for every book. The `{% empty %}` part is shown when there are no books.
`{{ forloop.counter }}`	Counts the rows: 1, 2, 3 and so on.
`{{ book.title }}`	Displays the title of the current book. The same idea works for every field.
`{{ ...|yesno:"Available,Issued" }}`	Turns True or False into the words Available or Issued.

Now open the home page of your website:

BROWSER **Type in the address bar**

```
http://127.0.0.1:8000/
```

CHECKPOINT The browser shows the heading **My Book Library**, the total count, and a table with all your books.

WHY WE DO THIS Templates use two kinds of Django code. `{{ ... }}` displays a value, and `{% ... %}` gives Django an instruction such as a loop. Everything else is ordinary HTML.

TIP **Try this now.** Go back to the admin page, add a sixth book, and refresh the library page. It appears without changing any code. This is what makes a website database-driven.

Part D: Adding Books from the Website

At the moment only the admin can add books. Now we add a form to our own website so that a visitor can add a book from a normal web page.

Step 15: Create the Form

Right-click the `books` folder, choose **New File**, name it `forms.py`, and type:

FILE **books/forms.py**

```
from django import forms
from .models import Book

class BookForm(forms.ModelForm):
    class Meta:
        model = Book
        fields = ['title', 'author', 'year_published', 'is_available']
```

WHY WE DO THIS A **ModelForm** builds an HTML form automatically from a model. `model = Book` says which model to use, and `fields` lists which columns appear in the form. Django even checks the data for you, for example that the year is a number.

WATCH OUT Save this file before you continue. If it is empty or unsaved, the next step gives the error `cannot import name 'BookForm'`. Also check that `BookForm` is spelled exactly the same everywhere.

Step 16: Add the Form View

Open `books/views.py`, delete everything, and type both views:

FILE **books/views.py**

```
from django.shortcuts import render, redirect
from .models import Book
from .forms import BookForm

def book_list(request):
    books = Book.objects.all()
    return render(request, 'books/book_list.html', {'books': books})

def add_book(request):
    if request.method == 'POST':
        form = BookForm(request.POST)
        if form.is_valid():
            form.save()
            return redirect('book_list')
    else:
        form = BookForm()
    return render(request, 'books/add_book.html', {'form': form})
```

The `add_book` view runs twice in the life of one form. Here is what happens:

1. The visitor clicks a link. The browser makes a **GET** request ("show me the page"). The code goes to `else` and creates an empty form: `form = BookForm()`.
2. The visitor fills in the form and clicks Save. The browser makes a **POST** request ("here is the data"). The code runs `BookForm(request.POST)` to load the data.
3. `form.is_valid()` checks the data. If it is fine, `form.save()` adds a new row to the database, and `redirect('book_list')` sends the visitor back to the library page.
4. If the data is not valid, the form is shown again with error messages.

> **WHY WE DO THIS** The same view handles both showing the empty form and receiving the filled form. The line `if request.method == 'POST'` is how it knows which of the two is happening.

Step 17: Add the URL for the Form

Open `books/urls.py`, delete everything, and type:

FILE **books/urls.py**

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.book_list, name='book_list'),
    path('add/', views.add_book, name='add_book'),
]
```

> **WHY WE DO THIS** Remember the pattern from Practice 1: every page needs a template, a view, and a URL. The view and URL are ready now. The template comes next.

Step 18: Create the Form Template

In the folder `books/templates/books/`, create a new file named `add_book.html` and type:

FILE **books/templates/books/add_book.html**

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Add Book</title>
</head>
<body>
    <h1>Add a New Book</h1>

    <form method="post">
        {% csrf_token %}
        {{ form.as_p }}
        <button type="submit">Save Book</button>
    </form>

    <p><a href="{% url 'book_list' %}">Back to library</a></p>
</body>
</html>
```

- `method="post"` tells the browser to send the data as a POST request.
- `{{ form.as_p }}` draws all the form fields, each inside its own paragraph.
- `{% url 'book_list' %}` builds the address of the library page from its nickname.

WHY WE DO THIS `{% csrf_token %}` adds a secret hidden code to the form. It proves the form really came from your site. Django refuses to accept forms without it, which protects the site from fake submissions.

WATCH OUT Never forget `{% csrf_token %}` inside a form. Without it, you get the error "CSRF verification failed".

Step 19: Add a Link to the Form

Open `book_list.html` and add this line directly below the line that shows the total books:

FILE **books/templates/books/book_list.html (add the second line)**

```
    <p>Total books: {{ books|length }}</p>
    <p><a href="{% url 'add_book' %}">+ Add a new book</a></p>
```

CHECKPOINT Refresh the library page and click **+ Add a new book**. Fill in the form and click **Save Book**. You return to the library page and the new book appears in the table.

TIP Try leaving the year empty and saving. The form refuses it and shows an error. That is `form.is_valid()` doing its job.

Summary: How It All Connects

File	Its job	Created in
`books/models.py`	Describes the Book table	Step 6
`db.sqlite3`	The database file that stores the books	Step 7
`books/admin.py`	Lets you manage books in the admin site	Step 9
`books/views.py`	Gets books from the database and handles the form	Steps 11, 16
`books/urls.py` and `config/urls.py`	Connect addresses to views	Steps 12, 13, 17
`books/templates/books/*.html`	The HTML pages	Steps 14, 18, 19
`books/forms.py`	Builds and checks the form	Step 15

Journey when someone opens the library page: Browser → `config/urls.py` → `books/urls.py` → `views.book_list` → `models.Book` → `db.sqlite3` → template → Browser.

Journey when someone adds a book: Form in browser → `views.add_book` → `forms.BookForm` checks the data → new row saved in `db.sqlite3` → redirect to the library page.

Practice Challenges

Try these on your own. Each one repeats something you learned.

1. **Add a genre column.** In `models.py`, add the line `genre = models.CharField(max_length=50, default='General')` inside the Book class. Stop the server (Ctrl + C), run `python manage.py makemigrations` and `python manage.py migrate`, and start the server again. Then add `'genre'` to `list_display` in `admin.py`, add `'genre'` to `fields` in `forms.py`, and add a Genre column (`<th>` and `<td>`) to `book_list.html`. Remember to change `colspan="5"` to `colspan="6"`.
2. **Sort the books.** In `views.py`, change `Book.objects.all()` to `Book.objects.order_by('title')` to list books alphabetically.
3. **Show only available books.** Change it to `Book.objects.filter(is_available=True)` and observe the result.
4. **Build your own idea.** Create a new project using the same steps for something you care about, such as a student marks register or a movie watchlist.

Common Errors and How to Fix Them

Read the **last line** of the error message in the terminal or browser, then find it below.

What you see	Likely cause	Fix
`(venv)` missing, or `No module named 'django'`	Virtual environment is not active	Run the Activate command from Step 2
`no such table: books_book`	The table was not created	Run `makemigrations` and then `migrate` (Step 7)
`does not appear to have any patterns`	`urlpatterns` is misspelled, or the file is not saved	Check the spelling and save the file
`cannot import name 'BookForm'`	`forms.py` is empty, unsaved, or the name is misspelled	Save `forms.py` and check the class name
`TemplateDoesNotExist`	Wrong template folder or file name	Path must be `books/templates/books/book_list.html`
`NoReverseMatch` for `add_book`	The URL was not added, or the name is different	Complete Step 17 and compare the names
`CSRF verification failed`	`{% csrf_token %}` is missing from the form	Add it inside the `<form>` tag
Book model missing in admin	Model not registered	Check `books/admin.py` (Step 9)
Library table is empty	No books added yet	Add books in the admin site (Step 10)
Cannot log in to admin	Wrong username or password	Run `python manage.py createsuperuser` again
`Address already in use`	An old server is still running	Press Ctrl + C in the old terminal, or use `runserver 8001`

Quick Command Reference

Task	Command
Activate the environment (Mac)	`source venv/bin/activate`
Activate the environment (Windows)	`venv\Scripts\activate`
Prepare a change to models	`python manage.py makemigrations`
Apply the change to the database	`python manage.py migrate`
Create an admin account	`python manage.py createsuperuser`
Start the server	`python manage.py runserver`
Stop the server	Press Ctrl + C in the terminal